

Secure SharePoint-to-Azure API

Architecture & Configuration Guide

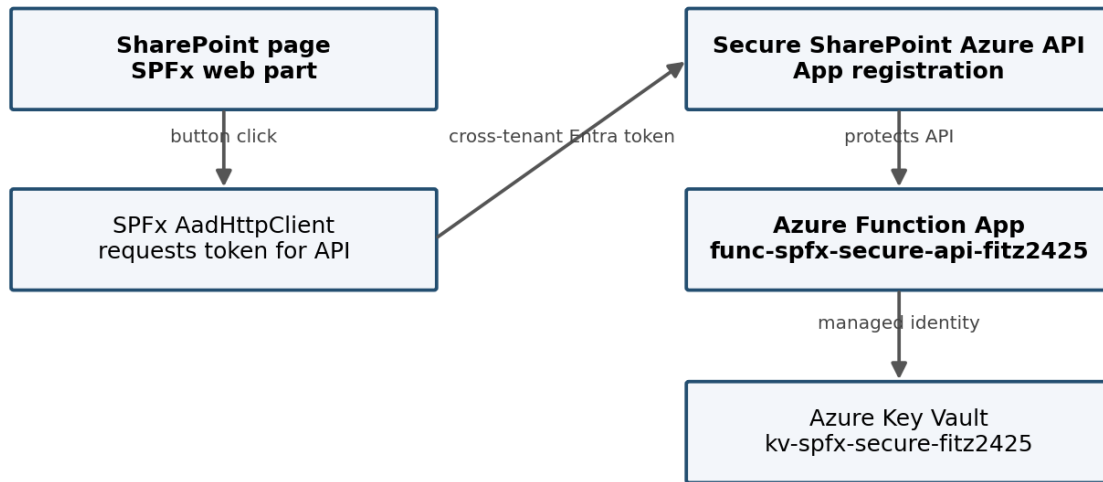
Cross-tenant SPFx -> Microsoft Entra ID -> Azure Function -> Key Vault

Microsoft 365 / SharePoint tenant

Tenant ID: 2a9ce2d2-ff9e-4e9b-a831-86b56b8daaa4

Azure tenant

Tenant ID: 36c396f9-b62a-46c0-b153-005205622cff



Two separate trust paths: user -> API (delegated token) and Function -> Key Vault (managed identity).

Environment documented: August 2026

What this document covers

The Azure resources, Microsoft Entra configuration, SharePoint/SPFx permission flow, CORS, Function App authentication, managed identity, Key Vault, Application Insights, and the end-to-end request path that successfully returned the authenticated user and Key Vault message.

Contents

1. 1. Executive overview
2. 2. Azure resources
3. 3. Cross-tenant Entra application setup
4. 4. SharePoint / SPFx permission configuration
5. 5. Function App authentication and CORS
6. 6. Managed identity and Key Vault
7. 7. End-to-end request flow
8. 8. Monitoring with Application Insights
9. 9. What each security layer is doing
10. 10. Troubleshooting lessons and final checklist

1. Executive overview

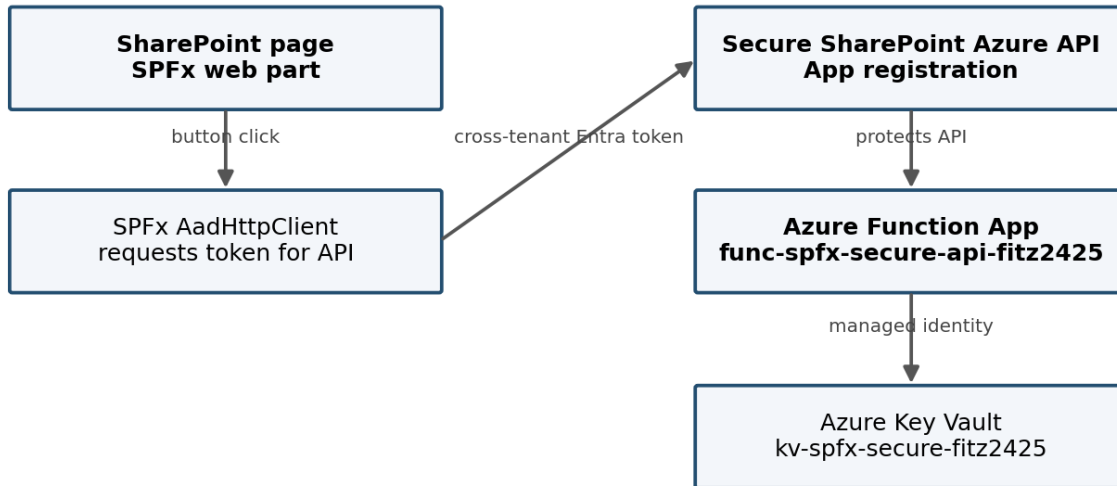
This solution lets a SharePoint Framework (SPFx) web part call a secured Azure Function. The web part runs in one Microsoft Entra tenant, while the Azure API is registered and hosted in a different Entra tenant. The API validates the signed-in SharePoint user, then uses the Function App's managed identity to read a value from Azure Key Vault.

Microsoft 365 / SharePoint tenant

Tenant ID: 2a9ce2d2-ff9e-4e9b-a831-86b56b8daaa4

Azure tenant

Tenant ID: 36c396f9-b62a-46c0-b153-005205622cff



Two separate trust paths: user -> API (delegated token) and Function -> Key Vault (managed identity).

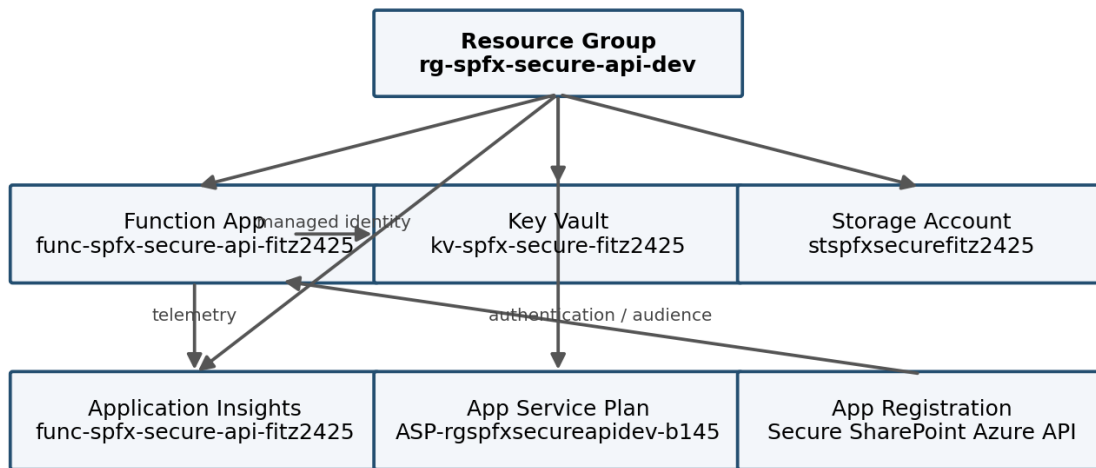
The two tenants

Purpose	Tenant ID	What lives there
Microsoft 365 / SharePoint	2a9ce2d2-ff9e-4e9b-a831-86b56b8daaa4	SharePoint Online, SPFx web part, signed-in user
Azure / API home tenant	36c396f9-b62a-46c0-b153-005205622cff	App registration, Function App, Key Vault, Application Insights, storage

Key architectural point

Because SharePoint and Azure are in different Entra tenants, this is a cross-tenant OAuth scenario. That is why the app had to be multitenant and why the SPFx package needed appId and replyUrl in webApiPermissionRequests.

2. Azure resources



Application Insights named appi-spfx-secure-api-dev also exists, but the Function is connected to the instance named func-spfx-secure-api-fitz2425.

Resource inventory

Resource	Name	Role in the solution
Resource Group	rg-spfx-secure-api-dev	Container for the Azure resources used by this solution.
Function App	func-spfx-secure-api-fitz2425	Hosts the secured HTTP API, including /api/profile.
Key Vault	kv-spfx-secure-fitz2425	Stores secrets/values outside source code.
Storage Account	stspfxsecurefitz2425	Required by Azure Functions for runtime/storage operations.
Application Insights	func-spfx-secure-api-fitz2425	Receives Function telemetry, requests, traces, failures, and Live Metrics.
App Service Plan	ASP-rgspfxsecureapidev-b145	Provides the compute plan that runs the Function App.
App Registration	Secure SharePoint Azure API	Defines the API identity, audience, exposed scope, multitenant behavior, and redirect URI.

Application Insights note

A second Application Insights resource named appi-spfx-secure-api-dev also exists. The Function App is connected to the Application Insights instance named func-spfx-secure-api-fitz2425, so that is the instance used for Live Metrics and request troubleshooting.

3. Cross-tenant Entra application setup

App registration identity

- Display name: Secure SharePoint Azure API
- Application (client) ID: [APPLICATION-ID-HIDDEN]
- Application ID URI: api://[APPLICATION-ID-HIDDEN]
- Delegated scope: access_as_user

Supported accounts

The app is configured for multiple Entra ID tenants, but it is restricted to specific tenants instead of being open to every tenant. The SharePoint tenant was explicitly added as an allowed tenant:

Allowed SharePoint tenant:
2a9ce2d2-ff9e-4e9b-a831-86b56b8daaa4

Redirect URI

The following Web redirect URI was added to the app registration and also used as the SPFx replyUrl:

<https://wffhz.sharepoint.com>

Front-channel logout URL was left blank. Implicit grant / hybrid flow checkboxes were left unchecked.

Why multitenant was required

The user signs in through the SharePoint tenant, not the Azure API home tenant. A single-tenant API would reject that user. Multitenant configuration allows Entra to issue a token for the API to a user whose identity originates in the SharePoint tenant.

4. SharePoint / SPFx permission configuration

The SPFx package requests delegated access to the API. Because the API is registered in another tenant, the request includes the API's appId and a replyUrl so SharePoint can complete the cross-tenant consent/service-principal flow.

```
"webApiPermissionRequests": [  
  {  
    "resource": "Secure SharePoint Azure API",  
    "scope": "access_as_user",  
    "appId": "[APPLICATION-ID-HIDDEN]",  
    "replyUrl": "https://wffhz.sharepoint.com"  
  }  
]
```

What each property means

Property	Meaning
resource	Human-readable name of the API SharePoint is requesting.
scope	Delegated permission exposed by the API:

	access_as_user.
appId	Globally unique application ID so SharePoint can identify the API across tenants.
replyUrl	Trusted return location used during the cross-tenant consent/registration flow.

Result

After these cross-tenant fields were added and the API was configured as multitenant, the SharePoint Admin Center was able to approve the Secure SharePoint Azure API / access_as_user request.

5. Function App authentication and CORS

Function App authentication

Azure App Service / Function authentication (Easy Auth) protects the Function before application code runs. The final configuration was adjusted for a cross-tenant token.

Setting	Final value / behavior
Issuer URL	https://login.microsoftonline.com/common/v2.0
Allowed token audience	api://[APPLICATION-ID-HIDDEN]
Tenant requirement	Allow requests from specific tenants
Allowed tenant	2a9ce2d2-ff9e-4e9b-a831-86b56b8daaa4
Client application requirement	Allow requests from any application
Identity requirement	Allow requests from any identity

CORS

Because SharePoint and the Function App are different web origins, the browser performs a CORS preflight before the API request. The Function App explicitly allows the SharePoint origin:

Allowed origin:

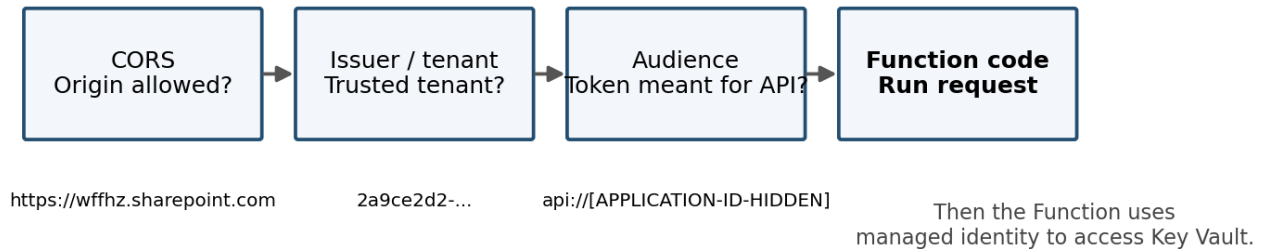
https://wffhz.sharepoint.com

Access-Control-Allow-Credentials: disabled

Why CORS is separate from authentication

CORS answers “may browser JavaScript from this website call me?” Authentication answers “is the caller presenting a valid token?” Both must succeed, but they solve different problems.

Inbound security checks before the Function executes



6. Managed identity and Key Vault

The Function App has a managed identity. This gives the Azure Function its own Entra identity without storing a client secret in source code.

Two identities are involved

- User identity: the SharePoint user is represented by the delegated Entra access token sent to the Function.
- Function identity: the Function App uses managed identity when it calls Azure Key Vault.

User -> SPFx -> Entra access token -> Azure Function

Azure Function -> Managed Identity -> Key Vault

Key Vault access

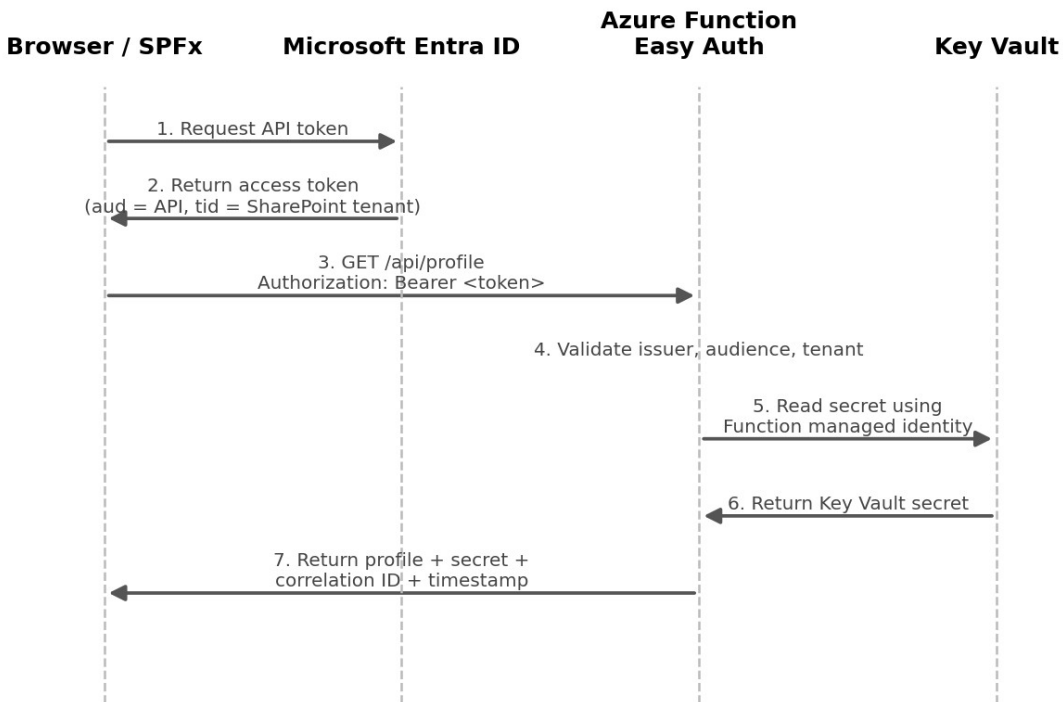
RBAC was configured so the Function App's managed identity can read the secret/value used by the API. The successful web-part response proved that this path works because the UI returned:

This message was retrieved securely from Azure Key Vault.

Security benefit

No Key Vault password or client secret needs to be embedded in SPFx, the Function source code, or GitHub. Azure issues short-lived credentials to the Function managed identity.

7. End-to-end request flow



CORS is enforced by the browser before the authenticated GET is allowed to proceed.

1. User clicks Call secured API in the SPFx web part.
2. SPFx AadHttpClient requests an Entra access token for api://[APPLICATION-ID-HIDDEN].
3. Entra issues a delegated token for the signed-in SharePoint user. The token is issued from the SharePoint tenant.
4. The browser sends the request to <https://func-spfx-secure-api-fitz2425.azurewebsites.net/api/profile> with Authorization: Bearer <token>.
5. CORS allows the SharePoint origin, then Easy Auth validates issuer/tenant and audience.
6. The Function runs and reads user claims such as name, email, and object ID.
7. The Function uses managed identity to retrieve the Key Vault value.
8. The Function returns the user profile, Key Vault message, correlation ID, and UTC timestamp to SPFx.

Successful result

The final test returned “Request succeeded” and displayed the authenticated user, email address, Entra object ID, Key Vault message, correlation ID, and timestamp. That result proves the full chain is functioning.

8. Monitoring with Application Insights

The Function App is connected to the Application Insights resource named func-spfx-secure-api-fitz2425. This is the monitoring resource to use during development and troubleshooting.

Live Metrics

- Watch incoming requests as they happen.
- See failed requests, exceptions, dependency activity, CPU, and memory.
- Use Sample Telemetry to inspect individual request/trace samples while testing.

Useful Logs queries

requests

| project timestamp, name, url, resultCode, duration, success
| order by timestamp desc
| take 20

traces

| order by timestamp desc
| take 50

Correlation IDs

The API response includes a correlation ID. Keeping that ID with the browser error makes it much easier to connect a user-visible failure to a specific request in Application Insights.

9. What each security layer is doing

Layer	Question it answers	Your configuration
SPFx API permission	May this SharePoint solution request access to the API?	access_as_user approved in SharePoint Admin Center
Multitenant app	May identities from the SharePoint tenant use this API?	SharePoint tenant explicitly allowed
Issuer / tenant validation	Who issued this token and is that tenant trusted?	common/v2.0 + allowed SharePoint tenant
Audience validation	Was this token actually issued for this API?	api://[APPLICATION-ID-HIDDEN]
CORS	May browser JavaScript from this site call the Function?	https://wffhz.sharepoint.com
Managed identity	Who is the Function when it calls another Azure service?	Function App system identity
Key Vault RBAC	May that Function identity read secrets?	RBAC grant to managed identity

Developer mental model

Authentication = who are you? Authorization = what may you do? CORS = which website may make the browser call? Managed identity = who is my Azure workload when it calls another Azure service?

10. Troubleshooting lessons and final checklist

Errors encountered and what they really meant

Error	Root cause / lesson
"The service principal ... could not be found"	The API was in a different Entra tenant. SPFx needed appId + replyUrl and the API needed multitenant configuration.
CORS: No Access-Control-Allow-Origin header	The Function App did not allow https://wffhz.sharepoint.com as an origin.
401 IDX10205 issuer validation failed	The Function auth provider was pinned to the API home tenant instead of accepting the SharePoint tenant issuer.
403 Forbidden after issuer fix	An additional Easy Auth authorization check was still too restrictive; the client application requirement was relaxed while tenant and audience checks remained in place.
Browse button returns 401	Expected for a secured API when the browser opens the Function without an Authorization bearer token.

Final configuration checklist

- SPFx package requests Secure SharePoint Azure API / access_as_user.
- webApiPermissionRequests includes appId and replyUrl for the cross-tenant API.
- App registration is multitenant and explicitly allows the SharePoint tenant.
- Web redirect URI is https://wffhz.sharepoint.com.
- Function CORS allows https://wffhz.sharepoint.com with credentials disabled.
- Function auth uses common/v2.0 issuer and validates the API audience.
- Function auth allows the SharePoint tenant.
- Function managed identity has Key Vault read access.
- Application Insights is connected and Live Metrics works.
- /api/profile returns authenticated user data and the Key Vault message.

Reusable pattern

This same secure middle-tier design can now be reused for Microsoft Graph, Jira, Workday, Yahoo Fantasy, databases, and AI services. The outer service changes; the SPFx -> Entra -> Azure Function security foundation stays largely the same.